

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    int i, sum = 0;
    for(i = 0; i <= n; i++) {
        sum = sum + arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {1,2,3,4,5}
    int total;

    total = sumArray(arr);

    if(total == 15) {
        printf("Correct Sum\n")
    }

    int a = 10, b = 0;
    int c = a / b;

    int *p;
    *p = 20;

    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2. The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
```

```
int main() {
    int lnum = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;
```

```
    printf("Welcome to Compiler Design);
```

```
    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;
```

```

char str1[] = "Hello;
char str2[] = "World";

int _valid = 10;
int invalid-id = 20;

float value = 5..6;
int data = 10@5;

char c = 'xy';
int 3value = 60;

printf("Value: %d\n", total);

return 0;
}

```

Tasks

- (a) Error Identification
 - i) Identify all lexical errors in the program.
 - ii) List the line number and the invalid token.
- (b) Error Classification
 - i) For each error, explain why it is considered a lexical error.
 - ii) Specify which rule of token formation is violated:
 - iii) Identifier rules
 - iv) Constant formation
 - v) String/character literal rules
 - vi) Operator/symbol rules
- (c) Token Correction
 - i) Rewrite each invalid token into a valid token.
 - ii) Example: 2value $\rightarrow$ value2
- (d) Compiler Behavior Analysis
 - i) Explain how a lexical analyzer (scanner) processes such errors.
 - ii) Discuss whether the compiler:
 - iii) Stops immediately
 - iv) Continues scanning using error recovery
- (e) Advanced Thinking
 - i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
 - ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

- A. Lexical Errors
- B. Syntax Errors
- C. Semantic Errors
- D. Intermediate Code Generation Issues (logic/precedence)
- E. Code Optimization Issues (redundancy/inefficiency)
- F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```

#include <stdio.h>

int divide(int a, int b) {
    return a / b
}

int main() {
    int #value = 20;
    float num = 15.5
    int result;

    result = divide(10);

    if(result = 5) {
        printf("Result is 5\n")
    }

    int x = 10;
    int y = 0;
    int z = x / y;

    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[4]);

    int *ptr;
    *ptr = 25;

    int total;
    total = x + y * result + 0;

    while(x < 20) {
        printf("%d\n", x);
    }

    return 0;
}

```

Do the following Tasks

(a) Identify Errors (Analysis Level)

- I. Identify **all errors** in the program.
- II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

- I. Lexical (invalid symbols like @num)
- II. Syntax (missing semicolon, brackets)
- III. Semantic (function argument mismatch)
- IV. Intermediate Code Issues (wrong expression handling/precedence)
- V. Optimization Issues (redundant + 0)
- VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

- I. Which errors are detected at **compile time**?
- II. Which errors appear only during **execution**?
- III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

- I. Rewrite the **fully corrected program**.
- II. Ensure:
 - A. Proper syntax
 - B. Correct semantics
 - C. Safe execution

(e) Deep Analysis (Evaluation)

- I. Explain how **incorrect pointer usage** affects runtime.
- II. Analyze how **array bounds violation** impacts memory.
- III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

- I. Suggest **2 improvements** in compiler design to:
 - A. Detect runtime errors earlier
 - B. Improve debugging messages
- II. Propose how **optimization phase** can remove redundant expressions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided

Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand